In [1]:
```
# import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import sqlite3
```

In [2]:
```
excel_file = 'customer_churn_data_raw.xlsx'
```

In [3]:
```
conn = sqlite3.connect('customer_churn.db')
```

In [4]:
```
excel_data = pd.ExcelFile(excel_file)
```

In [5]:
```
# Loop through each sheet and store as separate table
for sheet in excel_data.sheet_names:
    df = pd.read_excel(excel_file, sheet_name=sheet) # Read sheet into dataframe
    # Write dataframe to SQLite table
    df.to_sql(
        name=sheet,          # Table name = Sheet name
        con=conn,
        if_exists='replace', # Replace table if already exists
        index=False
    )

# Close connection
conn.close()

print("All sheets successfully converted into SQLite tables.")
```

```
All sheets successfully converted into SQLite tables.
```

In [9]:
```
# STEP1: IMPORT DATA
# Connect to SQLite database
conn = sqlite3.connect('customer_churn.db')

# sql query to Get all table names
sql_query = """
        SELECT name
        FROM sqlite_master
        WHERE type='table';
"""

# read sql query in pandas
tables = pd.read_sql(sql_query, conn)

# create dataframe for each table
for table_name in tables['name']:
    df = pd.read_sql(f"SELECT * FROM {table_name}", conn) # Read table into data
    globals()[f"df_{table_name}"] = df                    # Create dynamic dataf
    print(f"Created dataframe: df_{table_name}")

# Close connection
conn.close()
```

```
Created dataframe: df_db_customer
Created dataframe: df_db_subscription
Created dataframe: df_db_support
```

In [7]:
```
# Print table names and column names
conn = sqlite3.connect('customer_churn.db')

for table_name in tables['name']:
    print(f"\nTable Name: {table_name}")
    # Get column information
    columns_query = f"PRAGMA table_info({table_name});"
    columns = pd.read_sql(columns_query, conn)
    print("Columns:")
    print(columns['name'].tolist())

# Close connection
conn.close()
```

```
Table Name: db_customer
Columns:
['customerid', 'name', 'country', 'state', 'gender', 'dob', 'interests', 'pincod
e']

Table Name: db_subscription
Columns:
['customerid', 'subscription_start_date', 'subscription_type', 'renewal_date', 'p
lan_type', 'contract_type', 'cancellation_date', 'cancellation_reason', 'monthly_
charges', 'cltv', 'churn_score']

Table Name: db_support
Columns:
['customerid', 'complaint_date', 'escalations', 'csat_score', 'col_1', 'comment']
```

In [8]:
```
# STEP2: DATA CLEANING
```

In [14]:
```
# df_db_customer.head() # customer table
df_db_customer.tail()
```

Out[14]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>customerid</th> <th>customer_name</th> <th>country</th> <th>state</th> <th>gender</th> <th>dob</th> <th>interests</th> <th>pincode</th> </thead> <tbody> <th>16</th> <th>17</th> <th>18</th> <th>19</th> <th>20</th> </tbody> <tbody></tbody>

0020-JDNXP	rikim	India	Meghalaya	Female	1994-08-19 00:00:00	None	None
0021-IKXGC	vishakha	India	Rajasthan	Female	2000-09-02 00:00:00	None	None
0022-TCJCI	raghvendra	India	Telangana	Male	1983-12-30 00:00:00	None	None
0023-HGHWL	rishabh	India	Uttar Pradesh	Men	1991-05-14 00:00:00	None	None
0023-UYUPN	sudevi	India	Maharashtra	Women	1977-10-06 00:00:00	None	None

In [11]:
```
df_db_customer.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 8 columns):
 #   Column      Non-Null Count  Dtype 
---  ------      --------------  ----- 
 0   customerid  21 non-null     object
 1   name        21 non-null     object
 2   country     18 non-null     object
 3   state       21 non-null     object
 4   gender      21 non-null     object
 5   dob         21 non-null     object
 6   interests   4 non-null      object
 7   pincode     0 non-null      object
dtypes: object(8)
memory usage: 1.4+ KB
```

In [12]:
```
# a. rename col - name to customer_name
# b. drop columns - interest and pincode
# c. change data type - dob
# d. data standardization - gender
# e. fix missing values (using existing data) - country
```

In [13]:
```
# a. rename col - name to customer_name

df_db_customer.rename(columns = {'name' : 'customer_name'}, inplace= True)
```

In [15]:
```
# b. drop columns - interest and pincode

df_db_customer.drop(columns=['interests', 'pincode'], inplace=True)  # using col
```

In [16]:
```
df_db_customer.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype 
---  ------         --------------  ----- 
 0   customerid     21 non-null     object
 1   customer_name  21 non-null     object
 2   country        18 non-null     object
 3   state          21 non-null     object
 4   gender         21 non-null     object
 5   dob            21 non-null     object
dtypes: object(6)
memory usage: 1.1+ KB
```

In [17]:
```
# c. change data type - dob

df_db_customer['dob'] = pd.to_datetime(df_db_customer['dob'])
```

In [18]:
```
# d. data standardization - gender

# df_db_customer['gender'].unique()
df_db_customer['gender'] = df_db_customer['gender'].replace({'Men' : 'Male', 'Wo
```

In [19]:
```
df_db_customer['gender'].unique()
```

Out[19]:
```
array(['Male', 'Female'], dtype=object)
```

In [20]:
```
# e. fix missing values - country

# df_db_customer['country'].isna().sum()
df_db_customer[df_db_customer['country'].isna()]
```

Out[20]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>customerid</th> <th>customer_name</th> <th>country</th> <th>state</th> <th>gender</th> <th>dob</th> </thead> <tbody> <th>5</th> <th>8</th> <th>12</th> </tbody> <tbody></tbody>

0013-MHZWF	durga	None	Delhi	Female	1988-12-10
0015-UOCOJ	maya	None	Kathmandu	Female	1985-07-07
0018-NYROU	chitra	None	Telangana	Female	2004-12-01

In [21]:
```
# country and state - unique value pair

# Creating state → country map from non-null rows
state_country_mapping = df_db_customer.dropna(subset=['country']).set_index('sta

# Fill the missing country using State
df_db_customer['country'] = df_db_customer['country'].fillna(df_db_customer['sta
```

In [22]:
```
df_db_customer[df_db_customer['country'].isna()] # no null value in country col
```

Out[22]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>customerid</th> <th>customer_name</th> <th>country</th> <th>state</th> <th>gender</th> <th>dob</th> </thead> <tbody> </tbody> <tbody></tbody>

In [25]:
```
df_db_subscription.head() # subcription table
```

Out[25]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>customerid</th> <th>subscription_start_date</th> <th>subscription_type</th> <th>renewal_date</th> <th>plan_type</th> <th>contract_type</th> <th>cancellation_date</th> <th>cancellation_reason</th> <th>monthly_charges</th> <th>cltv</th> <th>churn_score</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody> </tbody>

0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	Annual	None	None	13.99	627	12
0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switched to competitor	12.99	1150	91
0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	Monthly	None	None	6.99	210	34
0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	Annual	None	None	22.99	1725	8
0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	Monthly	2024-02-28	Too expensive	13.99	195	88

In [26]:
```
df_db_subscription.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 11 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   customerid               21 non-null     object
 1   subscription_start_date  21 non-null     object
 2   subscription_type        21 non-null     object
 3   renewal_date             21 non-null     object
 4   plan_type                21 non-null     object
 5   contract_type            21 non-null     object
 6   cancellation_date        6 non-null      object
 7   cancellation_reason      6 non-null      object
 8   monthly_charges          21 non-null     float64
 9   cltv                     21 non-null     int64
 10  churn_score              21 non-null     int64
dtypes: float64(1), int64(2), object(8)
memory usage: 1.9+ KB
```

In [27]:
```
# change data type to date - subscription_start_date , renewal_date, cancellatio
date_col = ['subscription_start_date', 'renewal_date', 'cancellation_date']

df_db_subscription[date_col] = df_db_subscription[date_col].apply(pd.to_datetime
df_db_subscription.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 11 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   customerid               21 non-null     object
 1   subscription_start_date  21 non-null     datetime64[ns]
 2   subscription_type        21 non-null     object
 3   renewal_date             21 non-null     datetime64[ns]
 4   plan_type                21 non-null     object
 5   contract_type            21 non-null     object
 6   cancellation_date        6 non-null      datetime64[ns]
 7   cancellation_reason      6 non-null      object
 8   monthly_charges          21 non-null     float64
 9   cltv                     21 non-null     int64
 10  churn_score              21 non-null     int64
dtypes: datetime64[ns](3), float64(1), int64(2), object(5)
memory usage: 1.9+ KB
```

In [28]:
```python
df_db_support.head() # support table
```

Out[28]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>customerid</th> <th>complaint_date</th> <th>escalations</th> <th>csat_score</th> <th>col_1</th> <th>comment</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody> </tbody>

0003-MKNFE	2024-08-28 00:00:00	N	60	None	service issue
0003-MKNFE	2024-08-28 00:00:00	Y	10	None	demaned refund
0013-EXCHZ	2024-01-20 00:00:00	Y	20	None	None
0013-MHZWF	2025-03-18 00:00:00	N	90	None	guidance to renew
0013-SMEOE	2024-11-01 00:00:00	N	30	None	None

In [29]:
```python
df_db_support.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 6 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   customerid      9 non-null      object
 1   complaint_date  9 non-null      object
 2   escalations     9 non-null      object
 3   csat_score      9 non-null      int64
 4   col_1           0 non-null      object
 5   comment         4 non-null      object
dtypes: int64(1), object(5)
memory usage: 560.0+ bytes
```

In [30]:
```python
# drop columns
df_db_support.drop(columns=['col_1', 'comment'], inplace=True)
```

In [31]:
```python
# change data type to date - complaint_date
```

```
df_db_support['complaint_date'] = pd.to_datetime(df_db_support['complaint_date']
df_db_support.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 4 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   customerid      9 non-null      object
 1   complaint_date  9 non-null      datetime64[ns]
 2   escalations     9 non-null      object
 3   csat_score      9 non-null      int64
dtypes: datetime64[ns](1), int64(1), object(2)
memory usage: 416.0+ bytes
```

In [32]:
```
# Feature Engineering & Data Analysis
```

In [33]:
```
# create a new col using existing col - churn flag

# Customer is churned if cancellation_date is not null
df_db_subscription['churn_flag'] = np.where(df_db_subscription['cancellation_dat
df_db_subscription.head()
```

Out[33]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>customerid</th> <th>subscription_start_date</th> <th>subscription_type</th> <th>renewal_date</th> <th>plan_type</th> <th>contract_type</th> <th>cancellation_date</th> <th>cancellation_reason</th> <th>monthly_charges</th> <th>cltv</th> <th>churn_score</th> <th>churn_flag</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	Annual	NaT	None	13.99	627	12	0
0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switched to competitor	12.99	1150	91	1
0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	Monthly	NaT	None	6.99	210	34	0
0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	Annual	NaT	None	22.99	1725	8	0
0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	Monthly	2024-02-28	Too expensive	13.99	195	88	1

In [34]:
```
# first fix support table duplicates then merge
# Note: while merging df's always check the shape before and after
df = (df_db_subscription
            .merge(df_db_customer, on = 'customerid', how= 'left')
            .merge(df_db_support, on = 'customerid', how= 'left') )
```

In [35]:
```
df_db_subscription.shape
```

Out[35]: (21, 12)

In [36]:
```
df.shape
```

Out[36]: (23, 20)

In [37]:
```
print('df_db_subscription unique value:', df_db_subscription['customerid'].nuniq
print('df_db_customer unique value:', df_db_customer['customerid'].nunique())
print('df_db_support unique value:', df_db_support['customerid'].nunique())
print('df_db_support all value:', df_db_support['customerid'].size)
```

```
df_db_subscription unique value: 21
df_db_customer unique value: 21
df_db_support unique value: 7
df_db_support all value: 9
```

In [38]:
```
df_db_support
```

Out[38]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>customerid</th> <th>complaint_date</th> <th>escalations</th> <th>csat_score</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> </tbody> <tbody></tbody>

0003-MKNFE	2024-08-28	N	60
0003-MKNFE	2024-08-28	Y	10
0013-EXCHZ	2024-01-20	Y	20
0013-MHZWF	2025-03-18	N	90
0013-SMEOE	2024-11-01	N	30
0017-IUDMW	2024-04-10	Y	25
0019-EFAEP	2024-09-27	Y	30
0022-TCJCI	2024-09-13	Y	10
0022-TCJCI	2024-09-14	N	90

In [39]:
```
df_db_support['complaint_count'] = df_db_support.groupby('customerid')['customer
```

In [40]:
```
df_db_support = df_db_support.sort_values('complaint_date').drop_duplicates('cus
```

In [41]:
```
df_db_support['customerid'].size
```

Out[41]: 7

In [42]:
```
# merge df
df = (df_db_subscription
            .merge(df_db_customer, on = 'customerid', how= 'left')
            .merge(df_db_support, on = 'customerid', how= 'left') )
```

In [43]:
```
df.shape
```

Out[43]: (21, 21)

In [44]:
```
# export dataframe to csv file
df.to_csv('exported_churn_data.csv', index=False)
```

In [45]:
```
# DATA ANALYSIS
```

In [46]:
```
# 1. Churn Rate
```

In [47]:
```
churn_rate = df['churn_flag'].mean()*100
print("Churn Rate = ", round(churn_rate,2), "%")
```

```
Churn Rate =  28.57 %
```

In [48]:
```
# 2. Retenion Rate
retention_rate = 100 - churn_rate
print("Retention Rate = ", round(retention_rate,2), "%")
```

```
Retention Rate =  71.43 %
```

In [49]:
```
# 3. Churn by Plan type
churn_by_plan = (df.groupby('plan_type')['churn_flag'].mean().mul(100).round(2).
print(churn_by_plan)
```

```
  plan_type  churn_rate_pct
0     Basic           60.00
1   Premium           14.29
2  Standard           22.22
```

In [50]:
```
# 5. ARPU - Avg Revenue per user
arpu = df['monthly_charges'].mean()
print('ARPU = ', round(arpu,2))
```

```
ARPU =  18.85
```

In [51]:
```
# 6. Avg Customer Tenure
# count of days users has used our service : cancellation date else current date
today = pd.Timestamp.today()

df['tenure_days'] = np.where(
        df['cancellation_date'].notna(),

    (df['cancellation_date'] - df['subscription_start_date']).dt.days,

    (today - df['subscription_start_date']).dt.days
)

avg_tenure = df['tenure_days'].mean()
print("Avg Tenure (Days) = ", round(avg_tenure),0)
```

```
Avg Tenure (Days) =  1514 0
```

In [52]:
```
# 7. Revenue at risk - revenue lost from churned users
revenue_at_risk = df.loc[df['churn_flag']==1, 'monthly_charges'].sum()
print("Revenue at Risk (Rs 'K') =", revenue_at_risk)
```

```
Revenue at Risk (Rs 'K') = 73.94
```

In [53]:
```
# 8. Esclation Rate
escalation_rate = (df['escalations']=='Y').mean()*100
print("Esclation Rate = ", round(escalation_rate, 2), "%")
```

```
Esclation Rate =  19.05 %
```

In [54]:
```
# 9. Avg Complaint Per User
avg_complaints = df['complaint_count'].sum() / df['customerid'].nunique()
print("Avg Compliants Per User = ", round(avg_complaints, 2))
```

```
Avg Compliants Per User =  0.43
```

In [55]:
```
# 10. Correlation Esclation vs Churn
df['escalations'] = np.where(df['escalations'] == 'Y', 1, 0) # encoding string t
corr_df = df[['escalations', 'churn_flag']].dropna()

correlation = corr_df['escalations'].corr(df['churn_flag'])
print("Correlation between esclation vs churn is = ", round(correlation,2))
```

```
Correlation between esclation vs churn is =  0.77
```

In [60]:
```
# 11. Create a column using existing col - Churn risk
conditions = [
        (df['churn_score'] < 50),
        (df['churn_score'] >= 50) & (df['churn_score'] < 70),
        (df['churn_score'] >= 70)
]

choices = ['low', 'med', 'high']

df['churn_risk'] = np.select(conditions, choices, default='unkown')
```

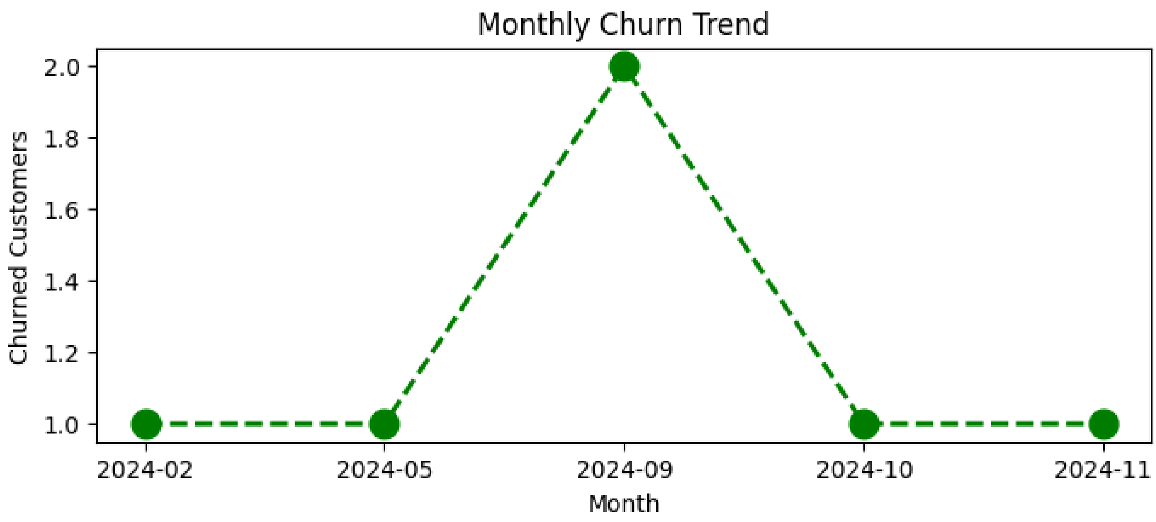
In [57]:
```
# Visualization using Matplotlib
```

In [58]:
```
# best practice to create a copy then work on it
df_visual = df.copy()
```

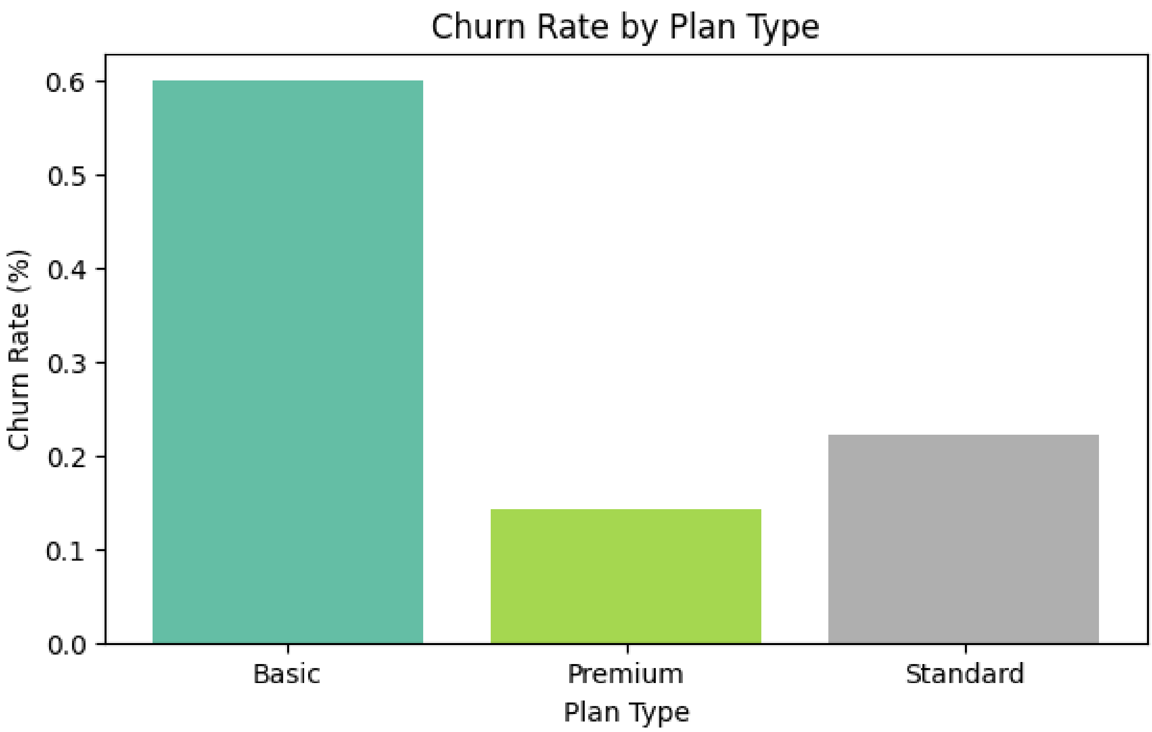
In [59]:
```
df_visual.columns
```

Out[59]:
```
Index(['customerid', 'subscription_start_date', 'subscription_type',
       'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
       'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
       'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
       'complaint_date', 'escalations', 'csat_score', 'complaint_count',
       'tenure_days', 'churn_risk'],
      dtype='object')
```

In [61]:
```
# 4.1 Monthly Churn Trend (Time Series KPI)

df_visual['cancellation_month'] = df_visual['cancellation_date'].dt.to_period('M

churn_trend = df_visual[df_visual['churn_flag'] == 1].groupby('cancellation_mont

plt.figure(figsize=(8,3))
plt.plot(churn_trend.index.astype(str), churn_trend.values,  color='green', mark

plt.title('Monthly Churn Trend')
plt.xlabel('Month')
plt.ylabel('Churned Customers')
plt.show()
```

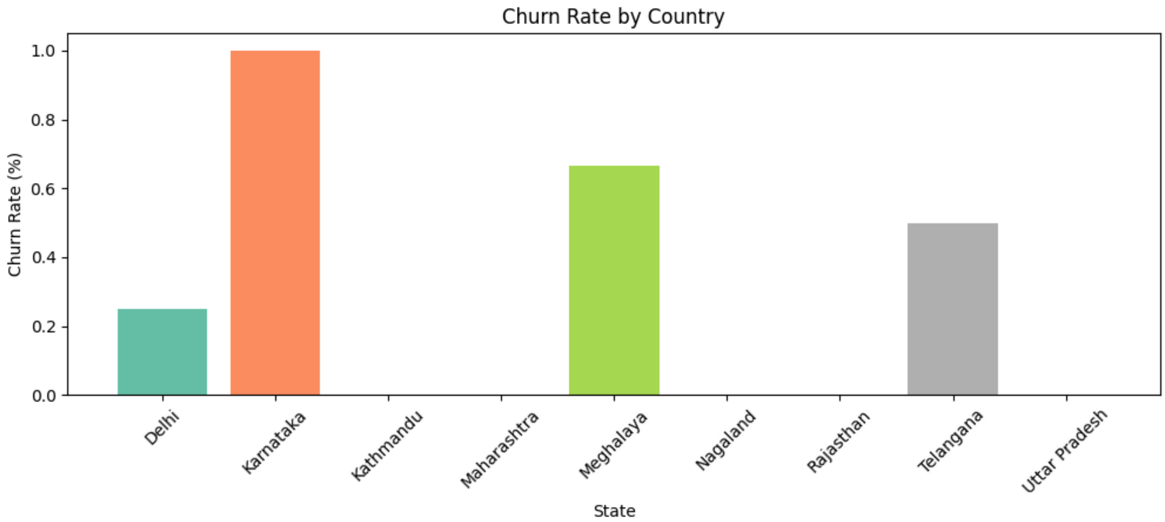


In [62]:
```
# 4.2 Churn Rate by Plan type
churn_plan = df_visual.groupby('plan_type')['churn_flag'].mean()

# colors = ['yellow', 'purple', 'blue']
colors = plt.cm.Set2(np.linspace(0, 1, len(churn_plan)))

plt.figure(figsize=(7,4))
plt.bar(churn_plan.index, churn_plan.values, color = colors)

plt.title('Churn Rate by Plan Type')
plt.xlabel('Plan Type')
plt.ylabel('Churn Rate (%)')
plt.show()
```



In [63]:
```
# 4.3 Churn by States
churn_plan = df_visual.groupby('state')['churn_flag'].mean()

# colors = ['yellow', 'purple', 'blue']
colors = plt.cm.Set2(np.linspace(0, 1, len(churn_plan)))
```

```
plt.figure(figsize=(12,4))
plt.bar(churn_plan.index, churn_plan.values, color = colors)

plt.title('Churn Rate by Country')
plt.xlabel('State')
plt.ylabel('Churn Rate (%)')
plt.xticks(rotation=45)
plt.show()
```



In [64]:
```
# Vsualization using seaborn
```

In [67]:
```
# encoding - convert str to numeric so that we can find corr between features
df_visual.columns
```

Out[67]:
```
Index(['customerid', 'subscription_start_date', 'subscription_type',
       'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
       'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
       'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
       'complaint_date', 'escalations', 'csat_score', 'complaint_count',
       'tenure_days', 'churn_risk', 'cancellation_month'],
      dtype='object')
```

In [66]:
```
df_visual[['plan_type', 'contract_type', 'churn_score', 'churn_flag', 'churn_ris
```

Out[66]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>plan_type</th> <th>contract_type</th> <th>churn_score</th> <th>churn_flag</th> <th>churn_risk</th> <th>escalations</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody> </tbody>

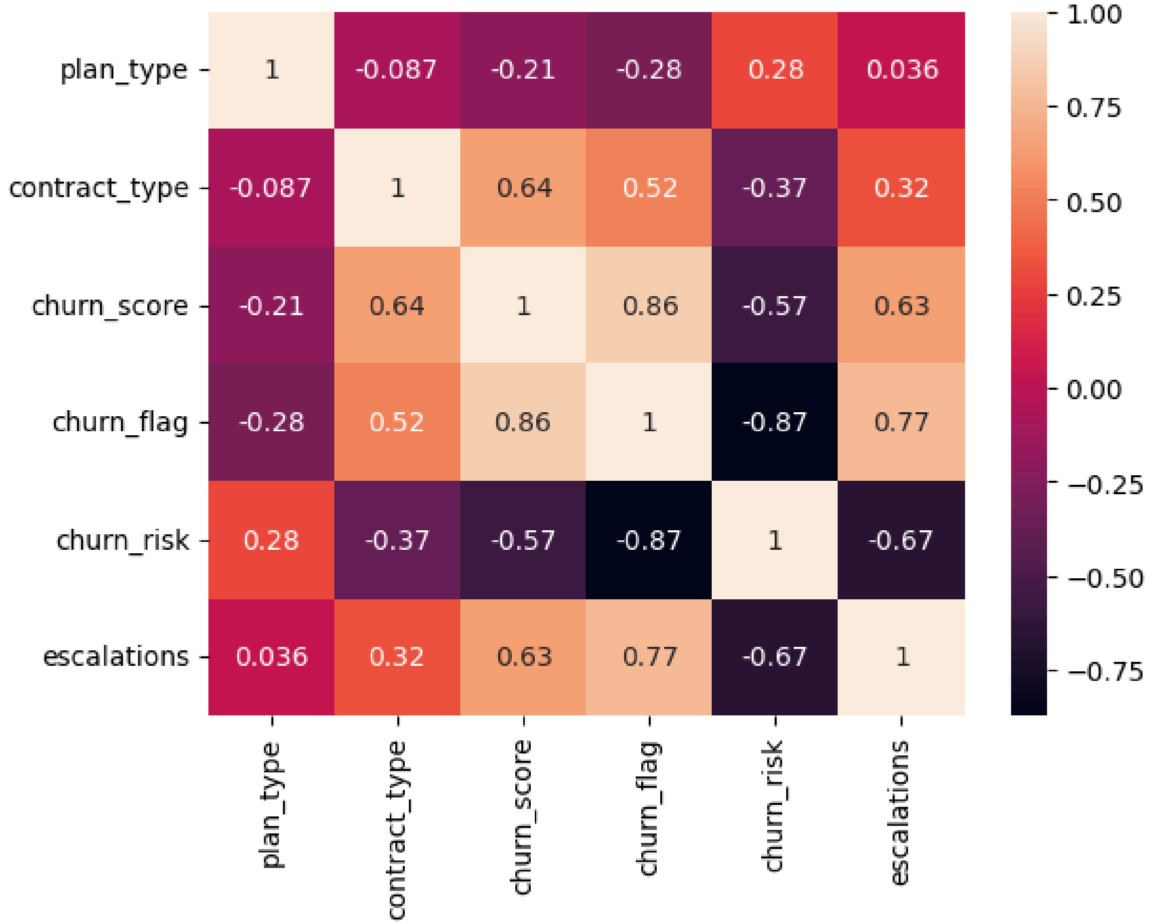
Standard	Annual	12	0	low	0
Premium	Annual	91	1	high	1
Basic	Monthly	34	0	low	0
Premium	Annual	8	0	low	0
Standard	Monthly	88	1	high	1

In [68]:
```python
# remove warnings
import warnings
warnings.filterwarnings("ignore")
```

In [69]:
```python
# incorrect method of encoding - as numbers are not assigned based on priority
df_encoded = df_visual[['plan_type', 'contract_type', 'churn_score', 'churn_flag

categorial_cols = ['plan_type', 'contract_type', 'churn_risk']

for col in categorial_cols:
    df_encoded[col] = df_encoded[col].astype('category').cat.codes
```

In [70]:
```python
# Heatmap (correlation matrix)

sns.heatmap(df_encoded.corr(), annot=True)
```

Out[70]: <Axes: >



In [71]:
```python
print('plan_type :', df_visual['plan_type'].unique())
print('contract_type :', df_visual['contract_type'].unique())
print('churn_risk :', df_visual['churn_risk'].unique())
```

```
plan_type : ['Standard' 'Premium' 'Basic']
contract_type : ['Annual' 'Monthly']
churn_risk : ['low' 'high' 'med']
```

In [72]:
```python
# Correct method of encoding - based on priority
df_encoded = df_visual[['plan_type', 'contract_type', 'churn_score', 'churn_flag

order_mappings = {
    'plan_type' : ['Basic', 'Standard', 'Premium'],
```

```
    'contract_type' : ['Monthly', 'Annual'],
    'churn_risk': ['low', 'med', 'high']
    }

for col, order in order_mappings.items():
    df_encoded[col] = pd.Categorical(df_encoded[col].astype('category'), categor
```

In [73]:

```
df_visual[['plan_type', 'contract_type', 'churn_score', 'churn_flag', 'churn_ris
```

Out[73]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>plan_type</th> <th>contract_type</th> <th>churn_score</th> <th>churn_flag</th> <th>churn_risk</th> <th>escalations</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody> </tbody>

Standard	Annual	12	0	low	0
Premium	Annual	91	1	high	1
Basic	Monthly	34	0	low	0
Premium	Annual	8	0	low	0
Standard	Monthly	88	1	high	1

In [74]:

```
df_encoded.head()
```

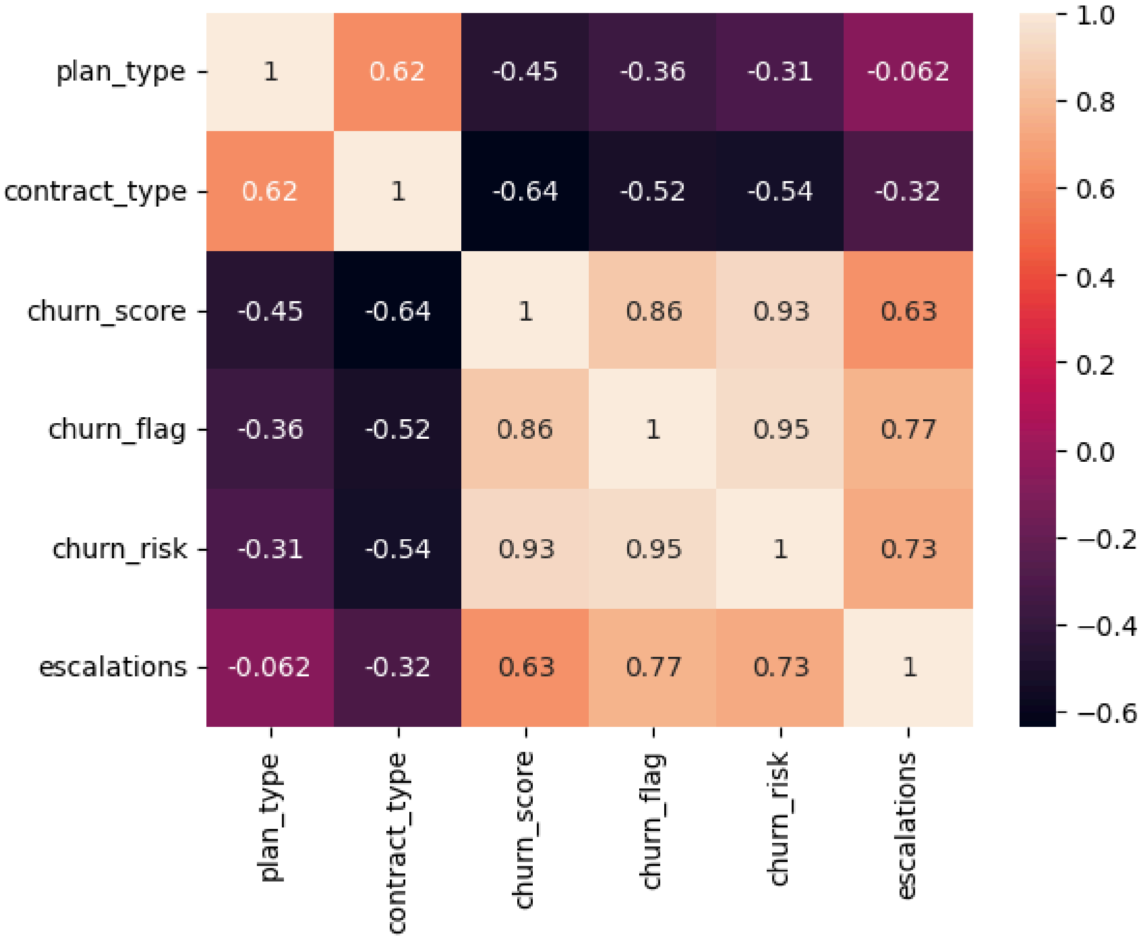
Out[74]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>plan_type</th> <th>contract_type</th> <th>churn_score</th> <th>churn_flag</th> <th>churn_risk</th> <th>escalations</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody> </tbody>

1	1	12	0	0	0
2	1	91	1	2	1
0	0	34	0	0	0
2	1	8	0	0	0
1	0	88	1	2	1

In [75]:

```
# Heatmap (correlation matrix)
sns.heatmap(df_encoded.corr(), annot=True)
```

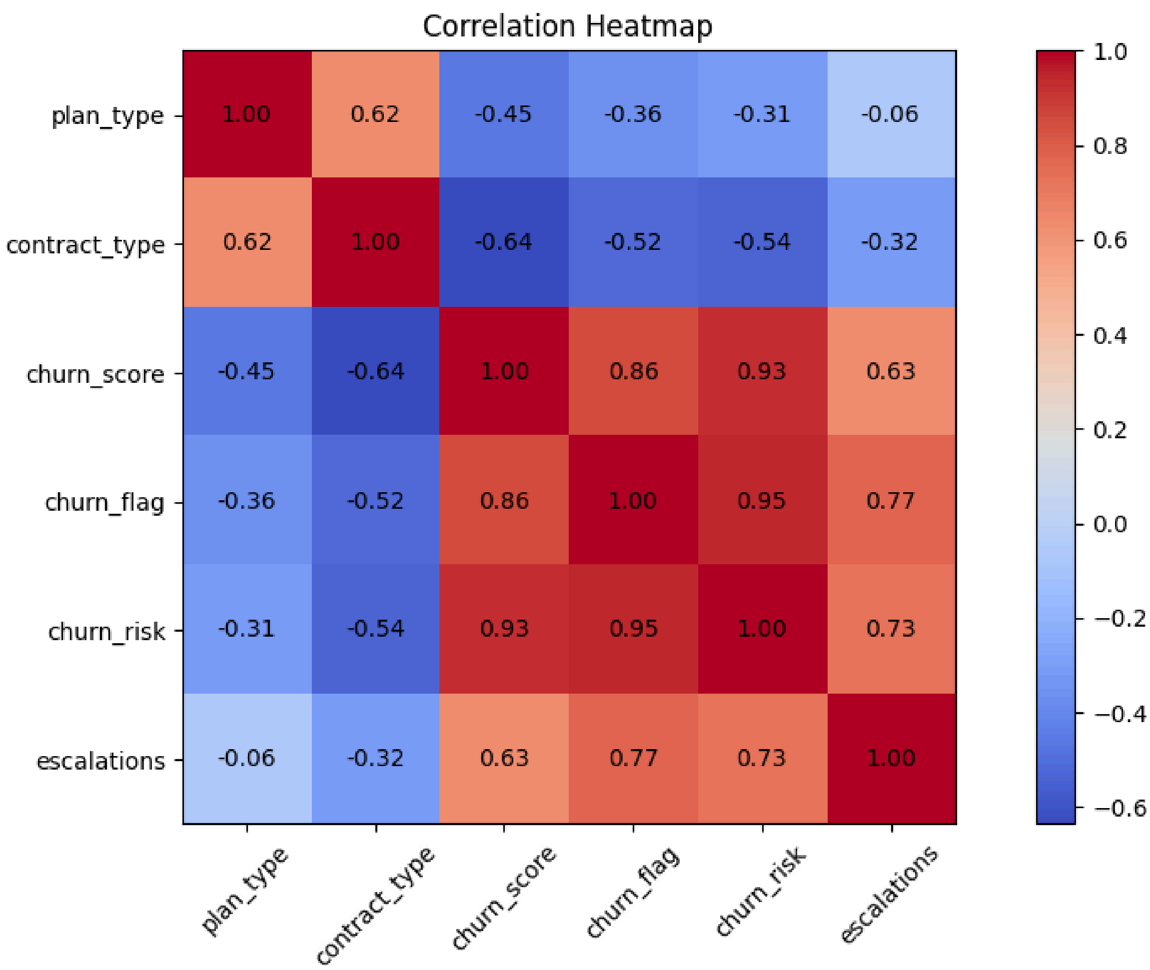
Out[75]: <Axes: >



In [76]:

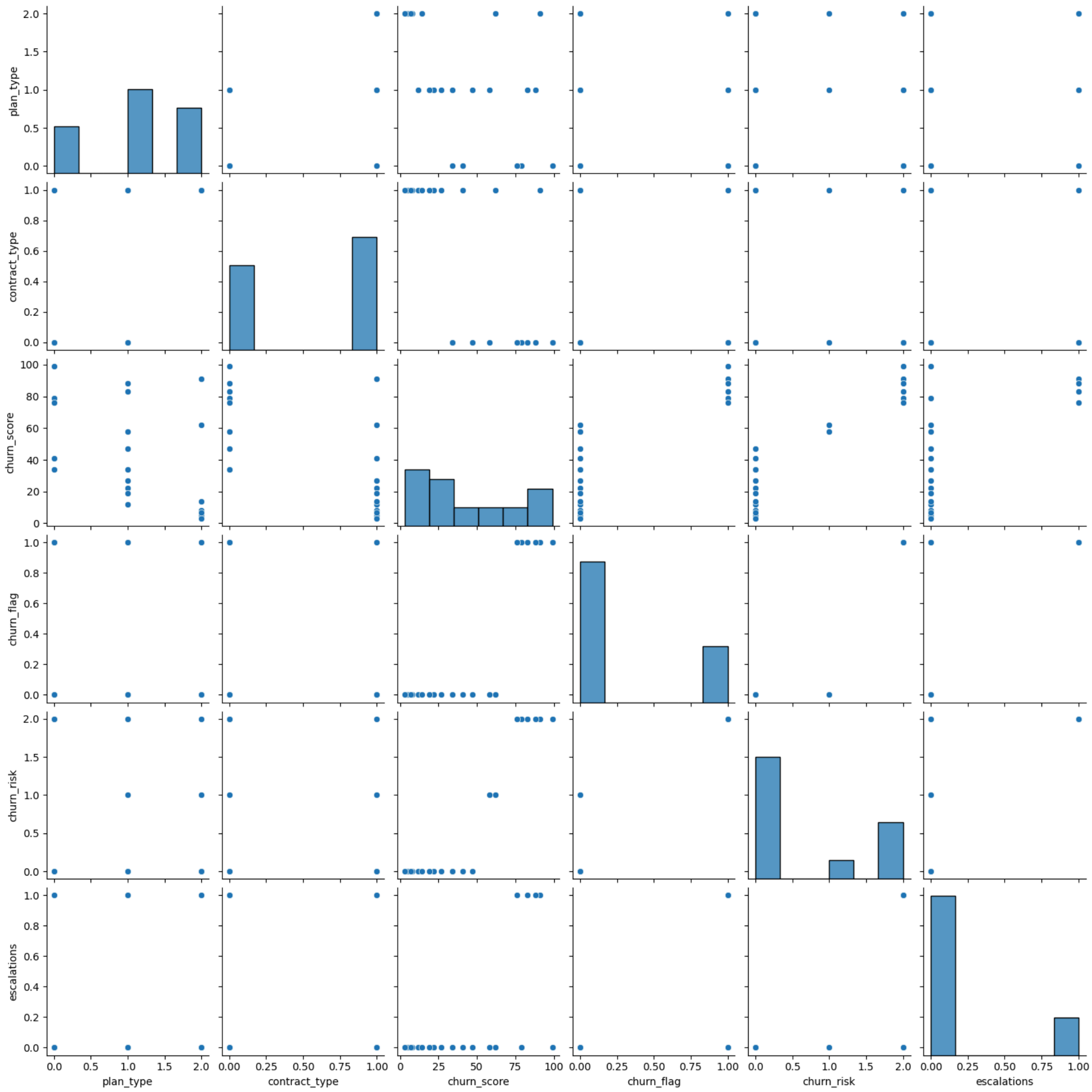
```python
# Heatmap Using Matplotlib - difficult to plot it

corr_matrix = df_encoded.corr() # Correlation matrix

fig, ax = plt.subplots(figsize=(10, 6)) # Create figure

cax = ax.imshow(corr_matrix, cmap='coolwarm') # Heatmap

fig.colorbar(cax) # Add colorbar

# Axis labels
ax.set_xticks(np.arange(len(corr_matrix.columns)))
ax.set_yticks(np.arange(len(corr_matrix.columns)))

ax.set_xticklabels(corr_matrix.columns, rotation=45)
ax.set_yticklabels(corr_matrix.columns)

# Annotate values inside cells
for i in range(len(corr_matrix.columns)):
    for j in range(len(corr_matrix.columns)):
        ax.text(
            j,
            i,
            f"{corr_matrix.iloc[i, j]:.2f}",
            ha='center',
            va='center'
        )

plt.title('Correlation Heatmap') # Title
```

```
plt.tight_layout()
plt.show()
```



In [77]:
```
# pairplot - Plot pairwise relationships in a dataset
sns.pairplot(df_encoded)
```

Out[77]: <seaborn.axisgrid.PairGrid at 0x265c9e708b0>

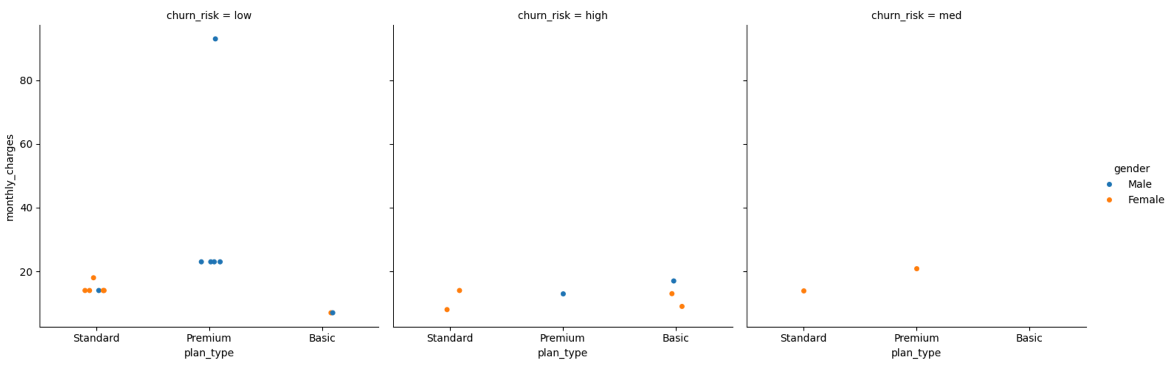


In [78]:
```
df_visual.columns
```

Out[78]:
```
Index(['customerid', 'subscription_start_date', 'subscription_type',
       'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
       'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
       'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
       'complaint_date', 'escalations', 'csat_score', 'complaint_count',
       'tenure_days', 'churn_risk', 'cancellation_month'],
      dtype='object')
```

In [79]:
```
# catplt/Facegrid plot - multi-dim comparison

sns.catplot(data=df_visual,
    x='plan_type',
    y='monthly_charges',
    hue='gender',
    col='churn_risk')
```

Out[79]:
```
<seaborn.axisgrid.FacetGrid at 0x265cb860760>
```



In [80]:

```
# Pivot Table

pd.pivot_table(
    df_visual,
    index='plan_type',
    values='churn_flag',
    aggfunc = 'mean'
)
```

Out[80]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>churn_flag</th> <th>plan_type</th> <th></th> </thead> <tbody> <th>Basic</th> <th>Premium</th> <th>Standard</th> </tbody> <tbody></tbody>

0.600000
0.142857
0.222222

In [81]:

```
# pivot table using multiple cols and agg type

pd.pivot_table(
    df_visual,
    index='plan_type',
    values=['monthly_charges', 'customerid', 'churn_flag'],
    aggfunc = {
        'monthly_charges' : 'sum',
        'customerid' : 'nunique',
        'churn_flag' : 'mean'
    }
)
```

Out[81]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>churn_flag</th> <th>customerid</th> <th>monthly_charges</th> <th>plan_type</th> <th></th> <th></th> <th></th> </thead> <tbody> <th>Basic</th> <th>Premium</th> <th>Standard</th> </tbody> <tbody></tbody>

0.600000	5	52.95
0.142857	7	218.93
0.222222	9	123.91

In [82]:
```
# working with SQL in Python
```

In [83]:
```
# create db in sql
conn = sqlite3.connect('test_database.sqlite')

#table details
conn.execute("CREATE TABLE users (first_name TEXT, country TEXT, budget INTEGER)

# commit and save
conn.commit()

print("Cretaed Table successfully!")
```

```
Cretaed Table successfully!
```

In [84]:
```
# insert data

cursor = conn.cursor()

# write sql query to insert records in sql table
cursor.execute(
    """
        INSERT INTO users VALUES
            ('Madhav', 'India', 5000),
            ('Rishabh', 'Germany', 2500),
            ('Vishakha', 'India', 3500)
    """
)

# commit and save
conn.commit()

print("Data Inserted successfully!")
```

```
Data Inserted successfully!
```

In [85]:
```
# check inserted data in table
conn = sqlite3.connect('test_database.sqlite')
query = """SELECT * FROM users"""

df_results = pd.read_sql(query, conn)

df_results.head()
```

Out[85]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>first_name</th> <th>country</th> <th>budget</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> </tbody> <tbody></tbody>

Madhav	India	5000
Rishabh	Germany	2500
Vishakha	India	3500

In [86]:
```
# aggregation
query = """
        SELECT country, SUM(budget) as total_budget
        FROM users
```

```
        GROUP BY country
"""
df_agg = pd.read_sql(query, conn)
df_agg
```

Out[86]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>country</th> <th>total_budget</th> </thead> <tbody> <th>0</th> <th>1</th> </tbody> <tbody></tbody>

Germany	2500
India	8500

In [87]:
```
conn.close()
```

In [ ]: